Why Node.js Uses a Thread Pool for File System Operations

First, Recall How Node.js is Supposed to Work

Before understanding the hack, you need to firmly understand what Node.js promises you normally.

Node.js says: **"I am non-blocking. I use a single thread. When I/O happens, I offload it to the Operating System, the OS handles it in the background, and when it is done the OS notifies me through the Event Loop."**

This is the whole selling point of Node.js. This is why Ryan Dahl built it. This is what makes it efficient.

The flow is supposed to be:

```
Your Code
   ↓
Node.js (single thread)
   ↓
Offload I/O to Operating System
   ↓
OS handles it in background (no thread needed)
   ↓
OS notifies Node.js when done (via epoll/kqueue/IOCP)
   ↓
Event Loop picks up the callback
   ↓
Your callback runs
```

No extra threads. Single thread. OS does the background work. Clean and efficient.

This works perfectly for network operations.

But file system operations are a different story entirely.

The Problem — OS Async Support for File System is Complicated

Here is the uncomfortable truth that the educator was explaining.

Network I/O vs File System I/O

Network I/O — things like making HTTP requests, database queries over a network, connecting to external APIs — the operating system has excellent, well-established, truly asynchronous support for these operations.

On Linux there is `epoll`. On macOS there is `kqueue`. On Windows there is `IOCP`. These are OS-level mechanisms that let you say "start this network operation, notify me when it is done" and the OS genuinely handles it in the background without needing a thread to sit and wait.

Network request → OS handles truly async → notifies Node.js → done
No thread needed. Genuinely non-blocking at OS level.

File System I/O — reading files, writing files, appending files, checking if files exist — this is where the problem lives.

The Ugly Truth About File System Async

The operating system's async support for file system operations is **inconsistent, incomplete, and unreliable** across different platforms.

Linux's `epoll` — which handles network async brilliantly — **does not fully support file system operations asynchronously.** Linux has a system called `aio` (Asynchronous I/O) for files, but it has severe limitations:

- It only works with files opened in a specific mode
- It has restrictions on buffer alignment
- It does not work with all file system types
- It behaves differently across different Linux kernel versions
- Many operations simply fall back to blocking anyway

macOS has similar inconsistencies. Windows IOCP handles files better but still has edge cases.

The bottom line is: **you cannot reliably tell the OS "read this file asynchronously, notify me when done" and trust that it will actually be non-blocking across all platforms and all situations.**

This is the exact problem the educator was describing when he said "fs async support nahi karta OS."

The Hack — libuv's Thread Pool

So Node.js has a problem. It promises non-blocking I/O. But the OS cannot reliably do file system operations in a truly async way.

libuv — the library at Node.js's core that handles the Event Loop and all async operations — solves this with what the educator called a "hack."

What the Hack Is

libuv maintains a pool of **background threads** specifically for operations that the OS cannot handle asynchronously.

This pool is called the **Thread Pool.**

By default it contains **4 threads.** You can configure it up to 1024 threads via an environment variable (`UV_THREADPOOL_SIZE`).

libuv Thread Pool (default 4 threads):

```
Thread 1  →  available for file operations
Thread 2  →  available for file operations
Thread 3  →  available for file operations
Thread 4  →  available for file operations
```

How the Hack Works

When you do a file system operation in Node.js — reading a file, writing a file, checking if a file exists — here is exactly what happens behind the scenes:

1. Your JavaScript code calls fs.readFile('./data.txt')

2. This goes to libuv

3. libuv checks: can the OS handle this truly async?
   Answer for file operations: NOT RELIABLY

4. libuv takes a thread from the thread pool
   (one of those 4 background threads)

5. That background thread calls the SYNCHRONOUS,
   BLOCKING version of the file read on the OS

6. That background thread BLOCKS, waiting for the
   file to be read — but crucially, it is a
   BACKGROUND THREAD, not your main JavaScript thread

7. Your main JavaScript thread is COMPLETELY FREE
   It continues running other code
   It handles other requests
   It does whatever it needs to do

8. The background thread finishes reading the file

9. libuv signals the Event Loop:
   "Hey, the file read is done, here is the data"

10. Event Loop picks this up when your main thread is free

11. Your callback runs with the file data

The Key Insight

From YOUR perspective as a developer — from the perspective of your JavaScript code — this feels completely asynchronous and non-blocking. Your main thread never blocks. You never wait. The Event Loop handles everything.

But from the perspective of what is actually happening in the computer — a real operating system thread is blocking and waiting for the file. It is just a different thread, hidden away in the background.

libuv is pretending to be async by using background threads to do the blocking work out of sight.

This is the hack. Fake async built on real blocking, but hidden from you.

Visualising the Difference

Network Request (Truly Async — No Extra Thread)

Main JS Thread:

```
────────────────────────────────────────────────────────────────────────────────
→ Start HTTP request
  OS takes over (epoll/kqueue/IOCP)
→ Handle other requests ←── main thread free
→ Handle other requests
→ Handle other requests
  OS: "HTTP response arrived!"
→ Run callback with response data
────────────────────────────────────────────────────────────────────────────────
```

No extra threads used. OS handles it natively.

File System Read (Thread Pool Hack)

Main JS Thread:

```
────────────────────────────────────────────────────────────────────────────────
→ Start fs.readFile()
  libuv grabs a background thread
→ Handle other requests ←── main thread free
→ Handle other requests
→ Handle other requests
```

Background Thread 1 (inside libuv):

```
────────────────────────────────────────────────────────────────────────────────
→ Calls blocking readFile on OS
  Waiting...
  Waiting...
  File read complete
→ Signals main thread via Event Loop
────────────────────────────────────────────────────────────────────────────────
```

Main JS Thread continues:

```
→ Event Loop: "file read done!"
→ Run your callback with file data
────────────────────────────────────────────────────────────────────────────────
```

Extra background thread used. Fake async.

Why Node.js is Still Called Single Threaded

This is the exact confusion the educator addressed when he said:

"Node.js single threaded hai — lekin diss is not 100% true"

The statement "Node.js is single threaded" refers specifically to **your JavaScript execution thread.** The thread that runs your JavaScript code — your callbacks, your functions, your logic — that is always single threaded.

But libuv underneath, the C++ layer that powers Node.js, absolutely uses multiple threads. The thread pool threads run in the background doing the blocking I/O work, but your JavaScript never directly interacts with them.

```
What you see and write:
┌──────────────────────────────────┐
│  JavaScript - Single Thread      │
│  Your code runs here             │
│  Event Loop runs here            │
│  Always single threaded          │
└──────────────────────────────────┘

What libuv does underneath:
┌──────────────────────────────────┐
│  libuv Thread Pool               │
│  Thread 1 (file ops)             │
│  Thread 2 (file ops)             │
│  Thread 3 (file ops)             │
│  Thread 4 (file ops)             │
│  Hidden from your JavaScript     │
└──────────────────────────────────┘
```

You never create these threads. You never manage them. You never see them. libuv handles all of that completely transparently.

What Operations Use the Thread Pool

It is not just file system operations. The thread pool is used for any operation where OS async support is unreliable or unavailable:

Uses Thread Pool (libuv handles):

- All `fs` module operations (readFile, writeFile, appendFile, etc.)
- DNS lookups (`dns.lookup`)
- Some crypto operations (computationally heavy ones)
- `zlib` compression/decompression
- User-defined C++ addons that need it

Does NOT Use Thread Pool (OS handles natively async):

- TCP/UDP network connections
- HTTP/HTTPS requests and servers
- Database connections over network
- `dns.resolve` (different from `dns.lookup`)
- Pipes and sockets

The Practical Implication — Thread Pool Exhaustion

Because the default thread pool only has 4 threads, if you make many simultaneous file system operations, they queue up waiting for a free thread.

```
// If you do this simultaneously:
fs.readFile('file1.txt', callback)  // gets thread 1
fs.readFile('file2.txt', callback)  // gets thread 2
fs.readFile('file3.txt', callback)  // gets thread 3
fs.readFile('file4.txt', callback)  // gets thread 4
fs.readFile('file5.txt', callback)  // WAITS for a free thread
fs.readFile('file6.txt', callback)  // WAITS for a free thread
```

Files 5 and 6 cannot start until one of the first four completes and releases its thread back to the pool.

This is why for applications that do heavy file system work, developers sometimes increase the thread pool size:

```
UV_THREADPOOL_SIZE=8 node app.js
```

Or in your code before anything runs:

process.env.UV_THREADPOOL_SIZE = 8

Summary — The Complete Picture

	Network I/O	File System I/O
OS Async Support	Excellent (epoll/kqueue/IOCP)	Incomplete and unreliable
How Node.js handles it	Offloads to OS directly	Uses thread pool (the hack)
Threads used	Zero extra threads	Background thread from pool
Main JS thread	Never blocked	Never blocked
Default capacity	Thousands simultaneously	4 simultaneously (default pool)
Truly async	Yes	No — fake async using threads

The educator's point was this: Node.js markets itself as non-blocking and single-threaded. For network operations, this is completely true. For file system operations, it is only true from your JavaScript perspective. Underneath, libuv is doing something much less elegant — using real blocking threads hidden in a pool — because the OS simply does not give a reliable alternative.

But crucially, **you do not need to worry about this.** From your code's perspective, `fs.readFile` is async and non-blocking. libuv handles all the complexity. The hack is completely invisible to you as a JavaScript developer. You write async code, you get async behaviour, and Node.js figures out how to actually deliver that behind the scenes.